

```

#include <windows.h>

#include <d3d11.h>

#include <d3dcompiler.h>

#include <cmath> // ИСПРАВЛЕНИЕ: Подключили математические функции sqrt и pow для CPU

#pragma comment(lib, "d3d11.lib")
#pragma comment(lib, "d3dcompiler.lib")

// === НОВЫЙ КОД: ИДЕНТИФИКАТОРЫ ДЛЯ ВЕРХНЕГО МЕНЮ WINDOWS ===
#define IDM_FILE_OPEN_VIDEO  1001 // Открыть видеофайл (AVI/MP4)
#define IDM_FILE_OPEN_IMAGE  1002 // Открыть отдельное изображение (BMP/PNG)
#define IDM_FILE_SAVE_RESULT 1003 // Записать преобразованный файл
#define IDM_FILE_EXIT        1004 // Выход из программы

#define IDM_VIEW_SPLIT        1005 // Режим "До / После" (разделение экрана)
#define IDM_VIEW_RESULT_ONLY 1006 // Режим "Только результат"

#define IDM_HELP_ABOUT        1007 // О программе
#define IDM_VIEW_CPU          40003

// Идентификаторы для дискретных масштабов дефектоскопа
#define IDM_ZOOM_FIT 40010
#define IDM_ZOOM_1   40011
#define IDM_ZOOM_2   40012
#define IDM_ZOOM_4    40013
#define IDM_ZOOM_8    40014
#define IDM_ZOOM_16   40015

// Структура вершины для нашего видеоэкрана
struct SimpleVertex
{
    float Pos[3]; // Координаты X, Y, Z в пространстве экрана
    float Tex[2]; // Текстурные координаты U, V для видеопотока

```

```
};
```

```
// Структура параметров кадра для передачи в шейдеры
```

```
struct ShaderConstants
```

```
{
```

```
    float width;    // Ширина картинки
```

```
    float height;   // Высота картинки
```

```
    float d_width;  // Шаг одного пикселя по горизонтали (1.0 / width)
```

```
    float d_height; // Шаг одного пикселя по вертикали (1.0 / height)
```

```
};
```

```
// Прототип функции программной фильтрации на центральном процессоре
```

```
HRESULT ApplyCpuFilter(UINT* pSrcPixels, UINT width, UINT height, UINT* pOutPixels);
```

```
// Глобальный указатель на буфер констант
```

```
ID3D11Buffer* g_pConstantBuffer = NULL;
```

```
// === НОВЫЙ КОД: ОБЪЕКТЫ ДЛЯ СВЕРТКИ АНАЛИЗА БЛОКОВ 4x4 ===
```

```
ID3D11RenderTargetView* g_pStage1RTV = NULL; // Холст для записи фиолетового анализа 3-го шейдера
```

```
ID3D11ShaderResourceView* g_pStage1SRV = NULL; // Ссылка на этот анализ для следующего шейдера
```

```
// === НОВЫЙ КОД: ОБЪЕКТЫ ДЛЯ УКРУПНЕННОГО АНАЛИЗА МАКРОБЛОКОВ 8x8 ===
```

```
ID3D11RenderTargetView* g_pStage2RTV = NULL; // Холст для записи макроанализа 4-го шейдера
```

```
ID3D11ShaderResourceView* g_pStage2SRV = NULL; // Ссылка на макроанализ для последующих шейдеров
```

```
// ОБЪЕКТЫ ДЛЯ ФИЛЬТРА ГЛОБАЛЬНОЙ КОНТРАСТНОСТИ (5-й ШЕЙДЕР)
```

```
ID3D11RenderTargetView* g_pStage3RTV = NULL; // Холст для записи HDR-фильтра
```

```
ID3D11ShaderResourceView* g_pStage3SRV = NULL; // Ссылка на этот результат
```

```
// === ТЕКСТУРНЫЕ БУФЕРЫ ДЛЯ СТАДИЙ ШЕЙДЕРНОГО СЖАТИЯ ===
```

```
ID3D11PixelShader* g_pPixelShaderDownsample = NULL; // Указатель на новый шейдер сжатия
```

```
// Стадия 1:2
```

```
ID3D11RenderTargetView* g_pStageDown2RTV = NULL;
```

```
ID3D11ShaderResourceView* g_pStageDown2SRV = NULL;
```

```
// Стадия 1:4
```

```
ID3D11RenderTargetView* g_pStageDown4RTV = NULL;
```

```
ID3D11ShaderResourceView* g_pStageDown4SRV = NULL;
```

```
// Стадия 1:8
```

```
ID3D11RenderTargetView* g_pStageDown8RTV = NULL;
```

```
ID3D11ShaderResourceView* g_pStageDown8SRV = NULL;
```

```
// Стадия 1:16
```

```
ID3D11RenderTargetView* g_pStageDown16RTV = NULL;
```

```
ID3D11ShaderResourceView* g_pStageDown16SRV = NULL;
```

```
// === НОВЫЕ ПЕРЕМЕННЫЕ ДЛЯ ПРОГРАММНОГО РЕЖИМА (CPU) ===
```

```
bool          g_bUseCPUProcessing = false; // true - процессор, false - видеокарта
```

```
ID3D11Texture2D* g_pCpuTexture = NULL;    // Промежуточный холст для пикселей от CPU
```

```
ID3D11ShaderResourceView* g_pCpuTextureSRV = NULL; // Ресурс чтения результата CPU для  
шторки
```

```
// === ГЛОБАЛЬНЫЕ ПЕРЕМЕННЫЕ ДЛЯ ПОЛОС ПРОКРУТКИ ===
```

```
int           g_scrollX = 0; // Текущий сдвиг картинки по горизонтали в пикселях
```

```
int           g_scrollY = 0; // Текущий сдвиг картинки по вертикали в пикселях
```

```
UINT          g_currentImgWidth = 0; // Храним размеры текущего файла
```

```
UINT          g_currentImgHeight = 0;
```

```
// === ГЛОБАЛЬНЫЕ РАЗМЕРЫ КЛИЕНТСКОЙ ОБЛАСТИ ОКНА ===
```

```
float         g_wndW = 800.0f; // Стартовая ширина окна по умолчанию
```

```
float         g_wndH = 600.0f; // Стартовая высота окна по умолчанию
```

```
// === ГЛОБАЛЬНЫЕ ДОЛИ ПРОКРУТКИ КАДРА (ОТ 0.0 ДО 1.0) ===
```

```
float          g_scrollRatioX = 0.0f;
```

```
float          g_scrollRatioY = 0.0f;
```

```
// Глобальный делитель масштаба кадра
```

```
float          g_zoomDivider = 1.0f;
```

```
// Глобальный флаг режима автоматического вписывания картинки в окно
```

```
bool           g_bFitToWindow = true; // По умолчанию плеер стартует в режиме "Вписать в кадр"!
```

```
// === ГЛОБАЛЬНЫЕ МАСШТАБНЫЕ МНОЖИТЕЛИ ДЛЯ ИДЕАЛЬНОГО СКРОЛЛИНГА ===
```

```
float          g_scrollMultiplierX = 1.0f;
```

```
float          g_scrollMultiplierY = 1.0f;
```

```
// === НОВЫЙ КОД: ПЕРЕМЕННАЯ ДЛЯ ХРАНЕНИЯ ПУТИ К ОТКРЫТОМУ ФАЙЛУ ===
```

```
wchar_t        g_szSelectedFilePath[MAX_PATH] = L""; // Буфер для пути к картинке или видео
```

```
// --- ГЛОБАЛЬНЫЕ ПЕРЕМЕННЫЕ (СТРОГО ПО ОДНОМУ ЭКЗЕМПЛЯРУ): ---
```

```
ID3D11Device* g_pd3dDevice = NULL;    // Видеокарта
```

```
ID3D11DeviceContext* g_pImmediateContext = NULL; // Контекст команд
```

```
IDXGISwapChain* g_pSwapChain = NULL;    // Буфер экрана
```

```
ID3D11RenderTargetView* g_pRenderTargetView = NULL; // Окно вывода
```

```
ID3D11VertexShader* g_pVertexShader = NULL; // Вершинный шейдер
```

```
ID3D11PixelShader* g_pPixelShader = NULL;    // Пиксельный шейдер
```

```
ID3D11PixelShader* g_pPixelShaderStage2 = NULL; // НОВОЕ: Указатель на 4-й шейдер анализа макроблоков 8x8
```

```
ID3D11PixelShader* g_pPixelShaderStage3 = NULL; // НОВОЕ: Указатель на 5-й шейдер адаптивной фильтрации контраста
```

```
ID3D11PixelShader* g_pPixelShaderFinal = NULL; // НОВОЕ: Указатель на 6-й шейдер финальной цветокоррекции и гаммы
```

```
ID3D11InputLayout* g_pVertexLayout = NULL;    // Формат вершин
```

```
ID3D11Buffer* g_pVertexBuffer = NULL;    // Буфер геометрии
```

```
ID3D11ShaderResourceView* g_pTextureSRV = NULL;    // Наша текстура 2x2
```

```
ID3D11SamplerState* g_pSamplerState = NULL;    // Жесткий сэмплер (Point)
```

```
LRESULT CALLBACK WindowProc(HWND hwnd, UINT uMsg, WPARAM wParam, LPARAM lParam);
```

```
void CleanupDevice(); // НОВОЕ: Объявляем функцию очистки заранее, чтобы убрать ошибку компиляции
```

```
// Функция для компиляции HLSL-файлов
```

```
HRESULT CompileShaderFromFile(const WCHAR* szFileName, LPCSTR szEntryPoint, LPCSTR szShaderModel, ID3DBlob** ppBlobOut)
```

```
{
```

```
    HRESULT hr = S_OK;
```

```
    ID3DBlob* pErrorBlob = NULL;
```

```
    hr = D3DCompileFromFile(szFileName, NULL, NULL, szEntryPoint, szShaderModel,
```

```
        D3DCOMPILE_ENABLE_STRICTNESS, 0, ppBlobOut, &pErrorBlob);
```

```
    // === ИСПРАВЛЕНИЕ: ТЕПЕРЬ ПЕРЕХВАТ НА СВОЕМ ЗАКОННОМ МЕСТЕ ВНУТРИ ФУНКЦИИ ===
```

```
    if (FAILED(hr))
```

```
    {
```

```
        if (pErrorBlob != NULL)
```

```
        {
```

```
            char* compileErrors = (char*)(pErrorBlob->GetBufferPointer());
```

```
            size_t newsize = strlen(compileErrors) + 1;
```

```
            wchar_t* wcstring = new wchar_t[newsize];
```

```
            size_t convertedChars = 0;
```

```
            mbstowcs_s(&convertedChars, wcstring, newsize, compileErrors, _TRUNCATE);
```

```
            // Показываем на экране точную строчную ошибку из файла shaders.hlsl
```

```
        MessageBoxW(NULL, wcstring, L"Критическая ошибка компиляции HLSL!", MB_OK |  
MB_ICONERROR);
```

```
        delete[] wcstring;  
        pErrorBlob->Release();  
    }  
    else  
    {  
        MessageBoxW(NULL, L"Файл shaders.hlsl не найден или поврежден!", L"Ошибка", MB_OK |  
MB_ICONERROR);  
    }  
    return hr;  
}  
  
return S_OK;  
}
```

```
// Инициализация видеокарты и создание объектов
```

```
HRESULT InitDevice(HWND hwnd)
```

```
{  
    HRESULT hr = S_OK;  
  
    RECT rc;  
    GetClientRect(hwnd, &rc);  
    UINT width = rc.right - rc.left;  
    UINT height = rc.bottom - rc.top;  
  
    DXGI_SWAP_CHAIN_DESC sd = {};  
    sd.BufferCount = 1;  
    sd.BufferDesc.Width = width;  
    sd.BufferDesc.Height = height;  
    sd.BufferDesc.Format = DXGI_FORMAT_R8G8B8A8_UNORM;  
    sd.BufferDesc.RefreshRate.Numerator = 60;
```

```
sd.BufferDesc.RefreshRate.Denominator = 1;

sd.BufferUsage = DXGI_USAGE_RENDER_TARGET_OUTPUT;

sd.OutputWindow = hwnd;

sd.SampleDesc.Count = 1;

sd.SampleDesc.Quality = 0;

sd.Windowed = TRUE;


D3D_DRIVER_TYPE driverType = D3D_DRIVER_TYPE_HARDWARE;

D3D_FEATURE_LEVEL featureLevel = D3D_FEATURE_LEVEL_11_0;


hr = D3D11CreateDeviceAndSwapChain(NULL, driverType, NULL, 0, &featureLevel, 1,
    D3D11_SDK_VERSION, &sd, &g_pSwapChain, &g_pd3dDevice, NULL, &g_pImmediateContext);
if (FAILED(hr)) return hr;


ID3D11Texture2D* pBackBuffer = NULL;

hr = g_pSwapChain->GetBuffer(0, __uuidof(ID3D11Texture2D), (LPVOID*)&pBackBuffer);
if (FAILED(hr)) return hr;


hr = g_pd3dDevice->CreateRenderTargetView(pBackBuffer, NULL, &g_pRenderTargetView);
pBackBuffer->Release();
if (FAILED(hr)) return hr;


g_pImmediateContext->OMSetRenderTargets(1, &g_pRenderTargetView, NULL);


D3D11_VIEWPORT vp;

vp.Width = (FLOAT)width;

vp.Height = (FLOAT)height;

vp.MinDepth = 0.0f;

vp.MaxDepth = 1.0f;

vp.TopLeftX = 0;

vp.TopLeftY = 0;

g_pImmediateContext->RSSetViewports(1, &vp);
```

```

// Компиляция Вершинного Шейдера
ID3DBlob* pVSBlob = NULL;

hr = CompileShaderFromFile(L"shaders.hlsl", "VS", "vs_4_0", &pVSBlob);

// === ИСПРАВЛЕНИЕ: ПЕРЕХВАТ ТЕКСТА ОШИБКИ ИЗ SHADERS.HLSL ===
if (FAILED(hr)) return hr; // Вернули стандартную проверку, ошибка C2065 исчезнет!


hr = g_pd3dDevice->CreateVertexShader(pVSBlob->GetBufferPointer(), pVSBlob->GetBufferSize(),
NULL, &g_pVertexShader);


if (FAILED(hr)) {
    MessageBoxW(hwnd, L"Ошибка на Шаге Ш-VS: Видеокарта отвергла Вершинный Шейдер!",
L"Сбой дефектоскопа", MB_OK | MB_ICONERROR);

    pVSBlob->Release();

    return hr;
}


// Описываем формат вершин для видеокарты (Input Layout)
D3D11_INPUT_ELEMENT_DESC layout[] =
{
    { "POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 0, D3D11_INPUT_PER_VERTEX_DATA, 0 },
    { "TEXCOORD", 0, DXGI_FORMAT_R32G32_FLOAT, 0, 12, D3D11_INPUT_PER_VERTEX_DATA, 0 },
};

UINT numElements = sizeof(layout) / sizeof(layout[0]);


hr = g_pd3dDevice->CreateInputLayout(layout, numElements, pVSBlob->GetBufferPointer(), pVSBlob-
>GetBufferSize(), &g_pVertexLayout);

pVSBlob->Release();

if (FAILED(hr)) return hr;


g_pImmediateContext->IASetInputLayout(g_pVertexLayout);

```

```

// Компиляция Пиксельного Шейдера

ID3DBlob* pPSBlob = NULL;

hr = CompileShaderFromFile(L"shaders.hlsl", "PS", "ps_4_0", &pPSBlob);

if (FAILED(hr)) return hr;


hr = g_pd3dDevice->CreatePixelShader(pPSBlob->GetBufferPointer(), pPSBlob->GetBufferSize(), NULL,
&g_pPixelShader);

pPSBlob->Release();

if (FAILED(hr)) return hr;


// === КОМПИЛЯЦИЯ НОВОГО ШЕЙДЕРА ПИРАМИДАЛЬНОГО СЖАТИЯ ===

hr = CompileShaderFromFile(L"shaders.hlsl", "PS_Downsample2X", "ps_4_0", &pBlob);

if (FAILED(hr)) return hr;


hr = g_pd3dDevice->CreatePixelShader(pBlob->GetBufferPointer(), pBlob->GetBufferSize(), NULL,
&g_pPixelShaderDownsample);

pBlob->Release();

if (FAILED(hr)) return hr;


// === НОВЫЙ КОД: КОМПИЛЯЦИЯ И СОЗДАНИЕ 4-ГО ШЕЙДЕРА (STAGE 2) ===

ID3DBlob* pPSStage2Blob = NULL;

// Указываем точку входа "PS_Stage2" — строго как имя функции в файле shaders.hlsl

hr = CompileShaderFromFile(L"shaders.hlsl", "PS_Stage2", "ps_4_0", &pPSStage2Blob);

if (FAILED(hr)) return hr;


hr = g_pd3dDevice->CreatePixelShader(pPSStage2Blob->GetBufferPointer(), pPSStage2Blob-
>GetBufferSize(), NULL, &g_pPixelShaderStage2);

pPSStage2Blob->Release(); // Освобождаем память временного буфера компиляции

if (FAILED(hr)) return hr;

// === КОНЕЦ НОВОГО КОДА ===


// === НОВЫЙ КОД: КОМПИЛЯЦИЯ И СОЗДАНИЕ 5-ГО ШЕЙДЕРА (STAGE 3) ===

ID3DBlob* pPSStage3Blob = NULL;

```

```

// Указываем точку входа "PS_Stage3" — строго как имя функции в файле shaders.hlsl
hr = CompileShaderFromFile(L"shaders.hlsl", "PS_Stage3", "ps_4_0", &pPSStage3Blob);
if (FAILED(hr)) return hr;

hr = g_pd3dDevice->CreatePixelShader(pPSStage3Blob->GetBufferPointer(), pPSStage3Blob->GetBufferSize(), NULL, &g_pPixelShaderStage3);

pPSStage3Blob->Release(); // Освобождаем память временного буфера компиляции
if (FAILED(hr)) return hr;

// === КОНЕЦ НОВОГО КОДА ===

// === НОВЫЙ КОД: КОМПИЛЯЦИЯ И СОЗДАНИЕ 6-ГО ШЕЙДЕРА (FINAL) ===
ID3DBlob* pPSFinalBlob = NULL;

// Указываем точку входа "PS_Final" — строго как имя функции в файле shaders.hlsl
hr = CompileShaderFromFile(L"shaders.hlsl", "PS_Final", "ps_4_0", &pPSFinalBlob);
if (FAILED(hr)) return hr;

hr = g_pd3dDevice->CreatePixelShader(pPSFinalBlob->GetBufferPointer(), pPSFinalBlob->GetBufferSize(), NULL, &g_pPixelShaderFinal);

pPSFinalBlob->Release(); // Освобождаем память временного буфера компиляции
if (FAILED(hr)) return hr;

// === КОНЕЦ НОВОГО КОДА ===

// Создаем тестовую текстуру 2x2 пикселя (Черный и Белый в шахматном порядке)
UINT textureData[] = {
    0xFFFFFFFF, 0xFF000000,
    0xFF000000, 0xFFFFFFFF
};

D3D11_TEXTURE2D_DESC desc = {};
desc.Width = 2;
desc.Height = 2;
desc.MipLevels = 1;
desc.ArraySize = 1;

```

```

desc.Format = DXGI_FORMAT_R8G8B8A8_UNORM;

desc.SampleDesc.Count = 1;

desc.Usage = D3D11_USAGE_DEFAULT;

desc.BindFlags = D3D11_BIND_SHADER_RESOURCE;


D3D11_SUBRESOURCE_DATA initData = {};

initData.pSysMem = textureData;

initData.SysMemPitch = 2 * sizeof(UINT);


ID3D11Texture2D* pTexture = NULL;

hr = g_pd3dDevice->CreateTexture2D(&desc, &initData, &pTexture);

if (SUCCEEDED(hr))
{
    hr = g_pd3dDevice->CreateShaderResourceView(pTexture, NULL, &g_pTextureSRV);

    pTexture->Release();
}

if (FAILED(hr)) return hr;


// Настраиваем режим POINT-фильтрации (строго без размытия)

D3D11_SAMPLER_DESC sampDesc = {};

sampDesc.Filter = D3D11_FILTER_MIN_MAG_MIP_POINT;

sampDesc.AddressU = D3D11_TEXTURE_ADDRESS_CLAMP;

sampDesc.AddressV = D3D11_TEXTURE_ADDRESS_CLAMP;

sampDesc.AddressW = D3D11_TEXTURE_ADDRESS_CLAMP;

sampDesc.ComparisonFunc = D3D11_COMPARISON_NEVER;

sampDesc.MinLOD = 0;

sampDesc.MaxLOD = D3D11_FLOAT32_MAX;


hr = g_pd3dDevice->CreateSamplerState(&sampDesc, &g_pSamplerState);

if (FAILED(hr)) return hr;


// === НОВЫЙ КОД: СОЗДАЕМ БУФЕР КОНСТАНТ НА ВИДЕОКАРТЕ ===

```

```

D3D11_BUFFER_DESC cbd = {};

cbd.Usage = D3D11_USAGE_DEFAULT;

cbd.ByteWidth = sizeof(ShaderConstants);    // Размер нашей структуры параметров

cbd.BindFlags = D3D11_BIND_CONSTANT_BUFFER; // Указываем видеокарте, что это буфер
констант

cbd.CPUAccessFlags = 0;


hr = g_pd3dDevice->CreateBuffer(&cbd, NULL, &g_pConstantBuffer);

if (FAILED(hr)) return hr;

// === КОНЕЦ НОВОГО КОДА ===


    // === НОВЫЙ КОД: СОЗДАЕМ СКРЫТУЮ ТЕКСТУРУ ДЛЯ РЕЗУЛЬТАТОВ 3-ГО ШЕЙДЕРА ===
D3D11_TEXTURE2D_DESC stage1Desc = {};

stage1Desc.Width = 2;                // Размер точно соответствует исходному кадру
stage1Desc.Height = 2;

stage1Desc.MipLevels = 1;

stage1Desc.ArraySize = 1;

stage1Desc.Format = DXGI_FORMAT_R8G8B8A8_UNORM; // Стандартный RGBA формат для
хранения данных

stage1Desc.SampleDesc.Count = 1;

stage1Desc.Usage = D3D11_USAGE_DEFAULT;

// Очень важные флаги: текстура будет одновременно холстом для записи и ресурсом для
чтения!

stage1Desc.BindFlags = D3D11_BIND_RENDER_TARGET | D3D11_BIND_SHADER_RESOURCE;


ID3D11Texture2D* pStage1Tex = NULL;

hr = g_pd3dDevice->CreateTexture2D(&stage1Desc, NULL, &pStage1Tex);

if (SUCCEEDED(hr))

{

    // Создаем "взгляд для записи" (RenderTargetView) — сюда 3-й шейдер будет писать данные
    g_pd3dDevice->CreateRenderTargetView(pStage1Tex, NULL, &g_pStage1RTV);


    // Создаем "взгляд для чтения" (ShaderResourceView) — отсюда 4-й шейдер будет их забирать

```

```

g_pd3dDevice->CreateShaderResourceView(pStage1Tex, NULL, &g_pStage1SRV);

pStage1Tex->Release(); // Саму текстуру видеокарта удержит внутри интерфейсов
}

if (FAILED(hr)) return hr;

// === КОНЕЦ НОВОГО КОДА ===

// === НОВЫЙ КОД: СОЗДАЕМ СКРЫТУЮ ТЕКСТУРУ ДЛЯ РЕЗУЛЬТАТОВ 4-ГО ШЕЙДЕРА ===
D3D11_TEXTURE2D_DESC stage2Desc = {};
stage2Desc.Width = 2;           // Размер по-прежнему соответствует тестовому кадру
stage2Desc.Height = 2;
stage2Desc.MipLevels = 1;
stage2Desc.ArraySize = 1;
stage2Desc.Format = DXGI_FORMAT_R8G8B8A8_UNORM; // Формат RGBA для хранения
макростатистики
stage2Desc.SampleDesc.Count = 1;
stage2Desc.Usage = D3D11_USAGE_DEFAULT;
stage2Desc.BindFlags = D3D11_BIND_RENDER_TARGET | D3D11_BIND_SHADER_RESOURCE;

ID3D11Texture2D* pStage2Tex = NULL;
hr = g_pd3dDevice->CreateTexture2D(&stage2Desc, NULL, &pStage2Tex);
if (SUCCEEDED(hr))
{
    // Создаем RenderTargetView — сюда 4-й шейдер запишет укрупненные данные 8x8
    g_pd3dDevice->CreateRenderTargetView(pStage2Tex, NULL, &g_pStage2RTV);

    // Создаем ShaderResourceView — отсюда данные заберет финальный сборочный шейдер
    g_pd3dDevice->CreateShaderResourceView(pStage2Tex, NULL, &g_pStage2SRV);

    pStage2Tex->Release();
}

if (FAILED(hr)) return hr;

// === КОНЕЦ НОВОГО КОДА ===

```

```

ID3D11Texture2D* pStage3Tex = NULL; // НОВОЕ: Объявляем указатель для третьей текстуры
// СОЗДАЕМ СКРЫТУЮ ТЕКСТУРУ ДЛЯ РЕЗУЛЬТАТОВ 5-ГО ШЕЙДЕРА
D3D11_TEXTURE2D_DESC stage3Desc = stage2Desc; // Копируем настройки формата и размеров
hr = g_pd3dDevice->CreateTexture2D(&stage3Desc, NULL, &pStage3Tex);

if (SUCCEEDED(hr))
{
    // Создаем RenderTargetView — сюда 5-й шейдер запишет результат фильтрации контраста
    hr = g_pd3dDevice->CreateRenderTargetView(pStage3Tex, NULL, &g_pStage3RTV);

    // Создаем ShaderResourceView — отсюда данные пойдут на финальный экран
    if (SUCCEEDED(hr))
    {
        hr = g_pd3dDevice->CreateShaderResourceView(pStage3Tex, NULL, &g_pStage3SRV);
    }

    pStage3Tex->Release(); // Передаем управление текстурой видеокарте
}

if (FAILED(hr)) return hr;

// Геометрия прямоугольного экрана по умолчанию (стартовая)
SimpleVertex vertices[] =
{
    { { -1.0f, 1.0f, 0.0f }, { 0.0f, 0.0f } },
    { { 1.0f, 1.0f, 0.0f }, { 1.0f, 0.0f } },
    { { -1.0f, -1.0f, 0.0f }, { 0.0f, 1.0f } },
    { { 1.0f, -1.0f, 0.0f }, { 1.0f, 1.0f } },
};

D3D11_BUFFER_DESC bd = {};
bd.Usage = D3D11_USAGE_DEFAULT;

```

```

    bd.ByteWidth = sizeof(SimpleVertex) * 4;

    bd.BindFlags = D3D11_BIND_VERTEX_BUFFER;

    bd.CPUAccessFlags = 0;


    D3D11_SUBRESOURCE_DATA InitData = {};

    InitData.pSysMem = vertices;


    hr = g_pd3dDevice->CreateBuffer(&bd, &InitData, &g_pVertexBuffer);

    if (FAILED(hr)) return hr;


    return S_OK;
}


// Обновленная функция отрисовки (Двухпроходный конвейер рендеринга)
// Обновленная функция отрисовки (Трехпроходный вычислительный конвейер)
// Обновленная функция отрисовки (Четырехпроходный вычислительный конвейер)
// Финальная функция отрисовки (Пятипроходный вычислительный конвейер)
// ПОЛНОЦЕННАЯ СИНХРОНИЗИРОВАННАЯ ФУНКЦИЯ ОТРИСОВКИ (ПЯТИПРОХОДНЫЙ КОНВЕЙЕР)
// ПРЯМОЙ СКВОЗНОЙ КОНВЕЙЕР: ИСКЛЮЧАЕТ ЛЮБЫЕ БЛОКИРОВКИ ТЕКСТУР В ПАМЯТИ
// ПОЛНОЦЕННАЯ СИНХРОНИЗИРОВАННАЯ ФУНКЦИЯ ОТРИСОВКИ С АВТОМАТИЧЕСКИМ
СКРОЛЛИНГОМ И СОХРАНЕНИЕМ ПРОПОРЦИЙ
// УТРЕННЯЯ ИСПРАВЛЕННАЯ ФУНКЦИЯ ОТРИСОВКИ С УМНЫМ ПОРТОМ ПРОСМОТРА
// ТОЧНАЯ КОПИЯ УТРЕННЕЙ РАБОЧЕЙ ФУНКЦИИ ОТРИСОВКИ (МЯГКИЕ ПОЛУТОНА И СЕРОЕ ПОЛЕ)
void Render()
{
    if (g_pRenderTargetView == NULL || g_pTextureSRV == NULL) return;


    // 1. Общие настройки сетки экрана
    UINT stride = sizeof(SimpleVertex);

    UINT offset = 0;

    g_plmmediateContext->IASetVertexBuffers(0, 1, &g_pVertexBuffer, &stride, &offset);

    g_plmmediateContext->IASetPrimitiveTopology(D3D11_PRIMITIVE_TOPOLOGY_TRIANGLESTRIP);

    g_plmmediateContext->IASetInputLayout(g_pVertexLayout);

```

```

// 2. Запрашиваем физические размеры кадра
ShaderConstants cbData;

cbData.width = 0.0f;

cbData.height = 0.0f;


ID3D11Resource* pResource = NULL;
g_pTextureSRV->GetResource(&pResource);
if (pResource)
{
    ID3D11Texture2D* pTex2D = (ID3D11Texture2D*)pResource;
    D3D11_TEXTURE2D_DESC desc;
    pTex2D->GetDesc(&desc);
    cbData.width = (float)desc.Width;
    cbData.height = (float)desc.Height;
    pResource->Release();
}


float originalFileW = cbData.width;
float originalFileH = cbData.height;


// Расчет шагов пикселя для промежуточных аналитических этапов (всегда в полный размер кадра!)
cbData.d_width = (cbData.width > 0.0f) ? (1.0f / cbData.width) : 0.0f;
cbData.d_height = (cbData.height > 0.0f) ? (1.0f / cbData.height) : 0.0f;
g_pImmediateContext->UpdateSubresource(g_pConstantBuffer, 0, NULL, &cbData, 0, 0);


// === ЖЕЛЕЗОБЕТОННАЯ МАТЕМАТИКА КОНВЕЙЕРА: ВПИСЫВАНИЕ И МАСШТАБЫ ПИКСЕЛЬ В ПИКСЕЛЬ ===


// 1. Стартовые текстурные координаты (вся картинка целиком)
float tLeft = 0.0f, tRight = 1.0f, tTop = 0.0f, tBottom = 1.0f;

// Стартовые геометрические рамки квадрата на экране (на все окно от -1.0 до +1.0)

```

```

float screenLeft = -1.0f, screenRight = 1.0f, screenTop = 1.0f, screenBottom = -1.0f;

if (originalFileW > 0.0f && originalFileH > 0.0f)
{
    // -----
    // ВАРИАНТ А: Режим "Вписать в кадр" (Автоматическое ужимание под размеры окна)
    // -----
    if (g_bFitToWindow)
    {
        // Вычисляем коэффициенты пропорций, чтобы лица не растягивались в "чудовище"!
        float ratioW = g_wndW / originalFileW;
        float ratioH = g_wndH / originalFileH;
        float minRatio = (ratioW < ratioH) ? ratioW : ratioH;

        float fitWidth = originalFileW * minRatio;
        float fitHeight = originalFileH * minRatio;

        // Сжимаем геометрические рамки квадрата строго под пропорции кадра, оставляя серое
поле!
        screenRight = -1.0f + 2.0f * (fitWidth / g_wndW);
        screenBottom = 1.0f - 2.0f * (fitHeight / g_wndH);
    }
    // -----
    // ВАРИАНТ Б: Честная дискретная масштабная сетка (1:1, 1:2, 1:4, 1:8, 1:16)
    // -----
    else
    {
        // Рассчитываем, сколько физических пикселей файла помещается в текущее окно
        float visiblePixelsX = g_wndW * g_zoomDivider;
        float visiblePixelsY = g_wndH * g_zoomDivider;

        // Если сжатый масштаб кадра больше физического окна — режим текстурное окно под
скроллинг долей

```

```

if (originalFileW > visiblePixelsX || originalFileH > visiblePixelsY)
{
    float viewW = (originalFileW > 0.0f) ? visiblePixelsX / originalFileW : 1.0f;
    float viewH = (originalFileH > 0.0f) ? visiblePixelsY / originalFileH : 1.0f;

    if (viewW > 1.0f) viewW = 1.0f;
    if (viewH > 1.0f) viewH = 1.0f;

    tLeft = (1.0f - viewW) * g_scrollRatioX;
    tRight = tLeft + viewW;
    tTop = (1.0f - viewH) * g_scrollRatioY;
    tBottom = tTop + viewH;
}

// Если сжатый кадр полностью поместился в окно — центрируем его и возвращаем серое
поле!
else
{
    screenRight = -1.0f + 2.0f * (originalFileW / (g_wndW * g_zoomDivider));
    screenBottom = 1.0f - 2.0f * (originalFileH / (g_wndH * g_zoomDivider));
}
}

// Собираем выверенную сетку вершин
SimpleVertex currentVertices[] =
{
    { { screenLeft, screenTop, 0.0f }, { tLeft, tTop } },
    { { screenRight, screenTop, 0.0f }, { tRight, tTop } },
    { { screenLeft, screenBottom, 0.0f }, { tLeft, tBottom } },
    { { screenRight, screenBottom, 0.0f }, { tRight, tBottom } },
};

if (g_pVertexBuffer)

```

```
{  
    g_plmmediateContext->UpdateSubresource(g_pVertexBuffer, 0, NULL, currentVertices, 0, 0);  
}
```

// Общие привязки конвейера

```
g_plmmediateContext->VSSetShader(g_pVertexShader, NULL, 0);  
g_plmmediateContext->PSSetConstantBuffers(0, 1, &g_pConstantBuffer);  
g_plmmediateContext->PSSetSamplers(0, 1, &g_pSamplerState);
```

```
ID3D11ShaderResourceView* nullSRV[] = { NULL };
```

```
float ClearColorBlack[] = { 0.0f, 0.0f, 0.0f, 1.0f };
```

```
float renderW = (originalFileW > 0.0f) ? originalFileW : 32.0f;
```

```
float renderH = (originalFileH > 0.0f) ? originalFileH : 32.0f;
```

```
// =====
```

```
// ДИАГНОСТИЧЕСКИЙ ПРОХОД: Расчет фильтров ВСЕГДА идет монолитно по всему файлу
```

```
// =====
```

```
g_plmmediateContext->ClearRenderTargetView(g_pStage3RTV, ClearColorBlack);
```

```
g_plmmediateContext->OMSetRenderTargets(1, &g_pStage3RTV, NULL);
```

```
D3D11_VIEWPORT vpS3 = {};
```

```
vpS3.Width = renderW;
```

```
vpS3.Height = renderH;
```

```
vpS3.MinDepth = 0.0f;
```

```
vpS3.MaxDepth = 1.0f;
```

```
g_plmmediateContext->RSSetViewports(1, &vpS3);
```

```
g_plmmediateContext->PSSetShader(g_pPixelShaderStage3, NULL, 0);
```

```
g_plmmediateContext->PSSetShaderResources(0, 1, &g_pTextureSRV);
```

```
g_plmmediateContext->PSSetShaderResources(1, 1, &g_pTextureSRV);
```

```
g_plmmediateContext->Draw(4, 0);
```

```

// =====
// ФИНАЛЬНЫЙ ПРОХОД: Вывод шторки на физический экран окна оператора
// =====

g_plmmediateContext->PSSetShaderResources(0, 1, nullSRV);
g_plmmediateContext->PSSetShaderResources(1, 1, nullSRV);

float ClearColorGrey[] = { 0.75f, 0.75f, 0.75f, 1.0f };
g_plmmediateContext->ClearRenderTargetView(g_pRenderTargetView, ClearColorGrey);
g_plmmediateContext->OMSetRenderTargets(1, &g_pRenderTargetView, NULL);

// ЗДЕСЬ ПОРТ ПРОСМОТРА ВСЕГДА РАВЕН КЛИЕНТСКОМУ ОКНУ ОКНА ОПЕРАТОРА
D3D11_VIEWPORT vpFull = {};
vpFull.Width = g_wndW;
vpFull.Height = g_wndH;
vpFull.MinDepth = 0.0f;
vpFull.MaxDepth = 1.0f;
vpFull.TopLeftX = 0;
vpFull.TopLeftY = 0;
g_plmmediateContext->RSSetViewports(1, &vpFull);

// Включаем 6-й шейдер финальной шторки сравнения
g_plmmediateContext->PSSetShader(g_pPixelShaderFinal, NULL, 0);

// Передаем в шейдер текущую ширину окна, чтобы шторка стояла ровно по центру экрана
окна!
cbData.width = g_wndW;
g_plmmediateContext->UpdateSubresource(g_pConstantBuffer, 0, NULL, &cbData, 0, 0);

if (g_bUseCPUProcessing && g_pCpuTextureSRV)
{
    g_plmmediateContext->PSSetShaderResources(0, 1, &g_pTextureSRV);
    g_plmmediateContext->PSSetShaderResources(1, 1, &g_pCpuTextureSRV);
}

```

```

    }
else
{
    g_pImmediateContext->PSSetShaderResources(0, 1, &g_pTextureSRV);
    g_pImmediateContext->PSSetShaderResources(1, 1, &g_pStage3SRV);
}
g_pImmediateContext->Draw(4, 0);

g_pImmediateContext->PSSetShaderResources(0, 1, nullSRV);
g_pImmediateContext->PSSetShaderResources(1, 1, nullSRV);

g_pSwapChain->Present(0, 0);
}

// === НОВЫЙ БЕЗОПАСНЫЙ КОД: ФУНКЦИЯ ДВУХЭКРАННОГО ВЫВОДА "ДО / ПОСЛЕ" ===
void RenderSplit()
{
    if (g_pRenderTargetView == NULL) return;

    // 1. Общие настройки сетки экрана
    UINT stride = sizeof(SimpleVertex);
    UINT offset = 0;

    g_pImmediateContext->IASetVertexBuffers(0, 1, &g_pVertexBuffer, &stride, &offset);
    g_pImmediateContext->IASetPrimitiveTopology(D3D11_PRIMITIVE_TOPOLOGY_TRIANGLESTRIP);
    g_pImmediateContext->IASetInputLayout(g_pVertexLayout);

    // 2. Получаем размеры оригинального файла для шейдеров
    ShaderConstants cbData;
    cbData.width = 2.0f;
    cbData.height = 2.0f;
    if (g_pTextureSRV)

```

```

{
    ID3D11Resource* pRes = NULL;
    g_pTextureSRV->GetResource(&pRes);
    if (pRes)
    {
        ID3D11Texture2D* pT2D = (ID3D11Texture2D*)pRes;
        D3D11_TEXTURE2D_DESC d;
        pT2D->GetDesc(&d);
        cbData.width = (float)d.Width;
        cbData.height = (float)d.Height;
        pRes->Release();
    }
}

cbData.d_width = 1.0f / cbData.width;
cbData.d_height = 1.0f / cbData.height;
g_plmmediateContext->UpdateSubresource(g_pConstantBuffer, 0, NULL, &cbData, 0, 0);

g_plmmediateContext->VSSetShader(g_pVertexShader, NULL, 0);
g_plmmediateContext->PSSetConstantBuffers(0, 1, &g_pConstantBuffer);
g_plmmediateContext->PSSetSamplers(0, 1, &g_pSamplerState);

ID3D11ShaderResourceView* nullSRV[] = { NULL };

// --- ВЫПОЛНЯЕМ СКРЫТЫЕ ПРОХОДЫ АНАЛИЗА (1, 2 и 3) ---
float ClearBlack[] = { 0.0f, 0.0f, 0.0f, 1.0f };

// Проход 1 (Stage1)
g_plmmediateContext->ClearRenderTargetView(g_pStage1RTV, ClearBlack);
g_plmmediateContext->OMSetRenderTargets(1, &g_pStage1RTV, NULL);
g_plmmediateContext->PSSetShader(g_pPixelShader, NULL, 0);
g_plmmediateContext->PSSetShaderResources(0, 1, &g_pTextureSRV);
g_plmmediateContext->Draw(4, 0);

```

```
// Проход 2 (Stage2)
```

```
g_plmmediateContext->PSSetShaderResources(0, 1, nullSRV);  
g_plmmediateContext->ClearRenderTargetView(g_pStage2RTV, ClearBlack);  
g_plmmediateContext->OMSetRenderTargets(1, &g_pStage2RTV, NULL);  
g_plmmediateContext->PSSetShader(g_pPixelShaderStage2, NULL, 0);  
g_plmmediateContext->PSSetShaderResources(0, 1, &g_pStage1SRV);  
g_plmmediateContext->Draw(4, 0);
```

```
// Проход 3 (Stage3)
```

```
g_plmmediateContext->PSSetShaderResources(0, 1, nullSRV);  
g_plmmediateContext->ClearRenderTargetView(g_pStage3RTV, ClearBlack);  
g_plmmediateContext->OMSetRenderTargets(1, &g_pStage3RTV, NULL);  
g_plmmediateContext->PSSetShader(g_pPixelShaderStage3, NULL, 0);  
g_plmmediateContext->PSSetShaderResources(0, 1, &g_pStage2SRV);  
g_plmmediateContext->Draw(4, 0);
```

```
// --- ВЫВОД НА ЭКРАН В ДВА ПОРТА ПРОСМОТРА ---
```

```
g_plmmediateContext->PSSetShaderResources(0, 1, nullSRV);  
float ClearGrey[] = { 0.75f, 0.75f, 0.75f, 1.0f };  
g_plmmediateContext->ClearRenderTargetView(g_pRenderTargetView, ClearGrey);  
g_plmmediateContext->OMSetRenderTargets(1, &g_pRenderTargetView, NULL);
```

```
// Жесткие безопасные размеры для деления окна пополам
```

```
float sideWidth = 400.0f;  
float sideHeight = 600.0f;
```

```
// ЛЕВАЯ ПОЛОВИНА: Сырой оригинал
```

```
D3D11_VIEWPORT vpL = {};  
vpL.Width = sideWidth;  
vpL.Height = sideHeight;  
vpL.MaxDepth = 1.0f;
```

```

g_pImmediateContext->RSSetViewports(1, &vpL);

g_pImmediateContext->PSSetShader(g_pPixelShader, NULL, 0);
g_pImmediateContext->PSSetShaderResources(0, 1, &g_pTextureSRV);
g_pImmediateContext->Draw(4, 0);
g_pImmediateContext->PSSetShaderResources(0, 1, nullSRV);

// ПРАВАЯ ПОЛОВИНА: Финальный сглаженный результат из Stage3
D3D11_VIEWPORT vpR = {};
vpR.Width = sideWidth;
vpR.Height = sideHeight;
vpR.TopLeftX = sideWidth; // Сдвиг вправо
vpR.MaxDepth = 1.0f;
g_pImmediateContext->RSSetViewports(1, &vpR);

g_pImmediateContext->PSSetShader(g_pPixelShaderFinal, NULL, 0);
g_pImmediateContext->PSSetShaderResources(0, 1, &g_pStage3SRV); // Выводим Stage3
g_pImmediateContext->Draw(4, 0);

// Сброс и вывод кадра
g_pImmediateContext->PSSetShaderResources(0, 1, nullSRV);
g_pSwapChain->Present(0, 0);
}

// Безопасное освобождение памяти при выходе
void CleanupDevice()
{
    if (g_pImmediateContext) g_pImmediateContext->ClearState();

    if (g_pStage1SRV) { g_pStage1SRV->Release(); g_pStage1SRV = NULL; }
    if (g_pStage1RTV) { g_pStage1RTV->Release(); g_pStage1RTV = NULL; }
    if (g_pStage2SRV) { g_pStage2SRV->Release(); g_pStage2SRV = NULL; } // НОВОЕ: Очистка ресурса
чтения Stage2

```

```

    if (g_pStage2RTV) { g_pStage2RTV->Release(); g_pStage2RTV = NULL; } // НОБОВЕ: Очистка холста
записи Stage2

    if (g_pStage3SRV) { g_pStage3SRV->Release(); g_pStage3SRV = NULL; }

    if (g_pStage3RTV) { g_pStage3RTV->Release(); g_pStage3RTV = NULL; }

    // Чистое освобождение ресурсов программного режима CPU

    if (g_pCpuTextureSRV) { g_pCpuTextureSRV->Release(); g_pCpuTextureSRV = NULL; }

    if (g_pCpuTexture) { g_pCpuTexture->Release(); g_pCpuTexture = NULL; }

    if (g_pSamplerState) { g_pSamplerState->Release(); g_pSamplerState = NULL; }

    if (g_pConstantBuffer) { g_pConstantBuffer->Release(); g_pConstantBuffer = NULL; }

    if (g_pTextureSRV) { g_pTextureSRV->Release(); g_pTextureSRV = NULL; }

    if (g_pVertexBuffer) { g_pVertexBuffer->Release(); g_pVertexBuffer = NULL; }

    if (g_pVertexLayout) { g_pVertexLayout->Release(); g_pVertexLayout = NULL; }

    if (g_pPixelShader) { g_pPixelShader->Release(); g_pPixelShader = NULL; }

    if (g_pPixelShaderStage2) { g_pPixelShaderStage2->Release(); g_pPixelShaderStage2 = NULL; } //
НОБОВЕ: Очистка 4-го шейдера анализа

    if (g_pPixelShaderStage3) { g_pPixelShaderStage3->Release(); g_pPixelShaderStage3 = NULL; } //
НОБОВЕ: Очистка 5-го шейдера фильтрации

    if (g_pPixelShaderFinal) { g_pPixelShaderFinal->Release(); g_pPixelShaderFinal = NULL; } // НОБОВЕ:
Очистка 6-го шейдера цветокоррекции

    if (g_pVertexShader) { g_pVertexShader->Release(); g_pVertexShader = NULL; }

    if (g_pRenderTargetView) { g_pRenderTargetView->Release(); g_pRenderTargetView = NULL; }

    if (g_pSwapChain) { g_pSwapChain->Release(); g_pSwapChain = NULL; }

    if (g_pImmediateContext) { g_pImmediateContext->Release(); g_pImmediateContext = NULL; }

    if (g_pd3dDevice) { g_pd3dDevice->Release(); g_pd3dDevice = NULL; }

}

```

// === НОВЫЙ КОД: ФУНКЦИЯ СОЗДАНИЯ СИСТЕМНОГО МЕНЮ ===

```
void CreateAppMenu(HWND hwnd)
```

```

{
    HMENU hMenu = CreateMenu();    // Главная горизонтальная полоса
    HMENU hFileMenu = CreatePopupMenu(); // Выпадающее меню "Файл"
    HMENU hViewMenu = CreatePopupMenu(); // Выпадающее меню "Вид"
    HMENU hHelpMenu = CreatePopupMenu(); // Выпадающее меню "Справка"
}

```

```

// Заполняем меню "Файл"

AppendMenuW(hFileMenu, MF_STRING, IDM_FILE_OPEN_VIDEO, L"Открыть видео поток
(AVI/MP4)...");

AppendMenuW(hFileMenu, MF_STRING, IDM_FILE_OPEN_IMAGE, L"Открыть кадр/картинку
(BMP/PNG)...");

// === НОВОЕ: ДОБАВЛЯЕМ УПРАВЛЕНИЕ РЕЖИМАМИ СРАВНЕНИЯ ===

AppendMenuW(hViewMenu, MF_STRING, IDM_VIEW_CPU, L"Программный режим (CPU)");

// === ВНЕДРЕНИЕ: КАСКАДНОЕ ПОДМЕНЮ ДИСКРЕТНЫХ МАСШТАБОВ ===

HMENU hZoomSubMenu = CreateMenu();

// Формируем 6 эталонных вариантов масштабирования

AppendMenuW(hZoomSubMenu, MF_STRING, IDM_ZOOM_FIT, L"Вписать в кадр");
AppendMenuW(hZoomSubMenu, MF_STRING, IDM_ZOOM_1, L"Масштаб 1:1 (Оригинал)");
AppendMenuW(hZoomSubMenu, MF_STRING, IDM_ZOOM_2, L"Масштаб 1:2");
AppendMenuW(hZoomSubMenu, MF_STRING, IDM_ZOOM_4, L"Масштаб 1:4");
AppendMenuW(hZoomSubMenu, MF_STRING, IDM_ZOOM_8, L"Масштаб 1:8");
AppendMenuW(hZoomSubMenu, MF_STRING, IDM_ZOOM_16, L"Масштаб 1:16");


// Встраиваем всплывающее подменю Масштаба внутрь нашего родительского меню "Вид"
[INDEX_146]

AppendMenuW(hViewMenu, MF_POPUP, (UINT_PTR)hZoomSubMenu, L"Масштаб");


AppendMenuW(hMenu, MF_POPUP, (UINT_PTR)hViewMenu, L"Вид");

AppendMenuW(hFileMenu, MF_SEPARATOR, 0, NULL); // Разделительная линия

AppendMenuW(hFileMenu, MF_STRING, IDM_FILE_SAVE_RESULT, L"Записать преобразованный
файл...");

AppendMenuW(hFileMenu, MF_SEPARATOR, 0, NULL);

AppendMenuW(hFileMenu, MF_STRING, IDM_FILE_EXIT, L"Выход");


// Заполняем меню "Вид"

AppendMenuW(hViewMenu, MF_STRING | MF_CHECKED, IDM_VIEW_RESULT_ONLY, L"Только
обработанный результат");

AppendMenuW(hViewMenu, MF_STRING, IDM_VIEW_SPLIT, L"Режим \"До / После\" (Разделение
экрана)");

```

```

// Заполняем меню "Справка"
AppendMenuW(hHelpMenu, MF_STRING, IDM_HELP_ABOUT, L"О программе...");

// Прикрепляем выпадающие списки к главной полосе меню
AppendMenuW(hMenu, MF_POPUP, (UINT_PTR)hFileMenu, L"Файл");
AppendMenuW(hMenu, MF_POPUP, (UINT_PTR)hViewMenu, L"Вид");
AppendMenuW(hMenu, MF_POPUP, (UINT_PTR)hHelpMenu, L"Справка");

// Физически вешаем меню на наше окно программы
SetMenu(hwnd, hMenu);
}

// === КОНЕЦ НОВОГО КОДА ===

#include <wincodec.h> // Подключаем заголовки системного декодера WIC

// === БЕЗОПАСНАЯ ЗАГРУЗКА ЛЮБЫХ КАРТИНОК ЧЕРЕЗ WIC НА ВИДЕОКАРТУ ===
HRESULT LoadTextureFromFile(const WCHAR* szFileName)
{
    HRESULT hr = S_OK;

    // 1. Создаем фабрику декодеров WIC
    IWICImagingFactory* pWICFactory = NULL;

    hr = CoCreateInstance(CLSID_WICImagingFactory, NULL, CLSCTX_INPROC_SERVER,
        __uuidof(IWICImagingFactory), (LPVOID*)&pWICFactory);
    if (FAILED(hr)) return hr;

    // 2. Открываем файл картинки на диске
    IWICBitmapDecoder* pDecoder = NULL;

    hr = pWICFactory->CreateDecoderFromFilename(szFileName, NULL, GENERIC_READ,
        WICDecodeMetadataCacheOnDemand, &pDecoder);
    if (FAILED(hr)) { pWICFactory->Release(); return hr; }
}

```

```

// 3. Берем самый первый кадр из файла
IWICBitmapFrameDecode* pFrame = NULL;
hr = pDecoder->GetFrame(0, &pFrame);
if (FAILED(hr)) { pDecoder->Release(); pWICFactory->Release(); return hr; }

// 4. Принудительно конвертируем пиксели в формат RGBA 32-бит
IWICFormatConverter* pConverter = NULL;
hr = pWICFactory->CreateFormatConverter(&pConverter);
if (FAILED(hr)) { pFrame->Release(); pDecoder->Release(); pWICFactory->Release(); return hr; }

hr = pConverter->Initialize(pFrame, GUID_WICPixelFormat32bppPBGRA, WICBitmapDitherTypeNone,
NULL, 0.0f, WICBitmapPaletteTypeCustom);

if (FAILED(hr)) { pConverter->Release(); pFrame->Release(); pDecoder->Release(); pWICFactory-
>Release(); return hr; }

// 5. Узнаем физические размеры загруженной картинки
UINT imgWidth = 0, imgHeight = 0;
pConverter->GetSize(&imgWidth, &imgHeight);

// Выделяем временный буфер в оперативной памяти компьютера под пиксели
UINT* pPixelsBuffer = new UINT[imgWidth * imgHeight];

hr = pConverter->CopyPixels(NULL, imgWidth * sizeof(UINT), imgWidth * imgHeight * sizeof(UINT),
(BYTE*)pPixelsBuffer);

if (SUCCEEDED(hr))
{
    // 6. Если старая основная текстура уже была в памяти — чисто освобождаем её
    if (g_pTextureSRV) { g_pTextureSRV->Release(); g_pTextureSRV = NULL; }

    // 7. Создаем новую текстуру прямо в видеопамяти под размеры нашей картинки
    D3D11_TEXTURE2D_DESC desc = {};
    desc.Width = imgWidth;
    desc.Height = imgHeight;

```

```

desc.MipLevels = 1;
desc.ArraySize = 1;
desc.Format = DXGI_FORMAT_R8G8B8A8_UNORM;
desc.SampleDesc.Count = 1;
desc.Usage = D3D11_USAGE_DEFAULT;
desc.BindFlags = D3D11_BIND_SHADER_RESOURCE;

D3D11_SUBRESOURCE_DATA initData = {};
initData.pSysMem = pPixelsBuffer;
initData.SysMemPitch = imgWidth * sizeof(UINT);

ID3D11Texture2D* pTexture = NULL;
hr = g_pd3dDevice->CreateTexture2D(&desc, &initData, &pTexture);
if (SUCCEEDED(hr))
{
    hr = g_pd3dDevice->CreateShaderResourceView(pTexture, NULL, &g_pTextureSRV);
    pTexture->Release();
}

// === НОВЫЙ КОД: ЗАПУСК ПРОГРАММНОЙ ОБРАБОТКИ CPU ПРИ ОТКРЫТИИ ФАЙЛА ===
if (SUCCEEDED(hr))
{
    // Выделяем память под массив пикселей, который обработает процессор
    UINT* pCpuOutPixels = new UINT[imgWidth * imgHeight];

    // Запускаем расчет на процессоре по алгоритмам 2011 года!
    if (SUCCEEDED(ApplyCpuFilter(pPixelsBuffer, imgWidth, imgHeight, pCpuOutPixels)))
    {
        // Описываем параметры для текстуры процессора
        D3D11_TEXTURE2D_DESC cpuDesc = {};
        cpuDesc.Width = imgWidth;
        cpuDesc.Height = imgHeight;
    }
}

```

```

cpuDesc.MipLevels = 1;
cpuDesc.ArraySize = 1;
cpuDesc.Format = DXGI_FORMAT_R8G8B8A8_UNORM;
cpuDesc.SampleDesc.Count = 1;
cpuDesc.Usage = D3D11_USAGE_DEFAULT;
cpuDesc.BindFlags = D3D11_BIND_SHADER_RESOURCE;

D3D11_SUBRESOURCE_DATA cpuInitData = {};
cpuInitData.pSysMem = pCpuOutPixels;
cpuInitData.SysMemPitch = imgWidth * sizeof(UINT);

// Физически создаем холст результатов CPU на видеокарте
ID3D11Texture2D* pCpuTex = NULL;
if (SUCCEEDED(g_pd3dDevice->CreateTexture2D(&cpuDesc, &cpuInitData, &pCpuTex)))
{
    g_pd3dDevice->CreateShaderResourceView(pCpuTex, NULL, &g_pCpuTextureSRV);
    pCpuTex->Release();
}

// Освобождаем временный массив результатов процессора из ОЗУ
delete[] pCpuOutPixels;
}

// =====
// СИНХРОНИЗАЦИЯ С КОДОМ 2011 ГОДА: ДИНАМИЧЕСКИЙ РЕЗАЙЗ СКРЫТЫХ БУФЕРОВ
// =====
if (SUCCEEDED(hr))
{
    // Чисто освобождаем все старые скрытые буферы из видеопамати [INDEX_45, INDEX_46]
    if (g_pStage1RTV) { g_pStage1RTV->Release(); g_pStage1RTV = NULL; }
    if (g_pStage1SRV) { g_pStage1SRV->Release(); g_pStage1SRV = NULL; }
}

```

```

if (g_pStage2RTV) { g_pStage2RTV->Release(); g_pStage2RTV = NULL; }
if (g_pStage2SRV) { g_pStage2SRV->Release(); g_pStage2SRV = NULL; }
if (g_pStage3RTV) { g_pStage3RTV->Release(); g_pStage3RTV = NULL; }
if (g_pStage3SRV) { g_pStage3SRV->Release(); g_pStage3SRV = NULL; }

// Вычисляем правильные пирамидальные размеры сжатия [INDEX_79]
UINT stage1Width = (imgWidth / 4 > 0) ? imgWidth / 4 : 1;
UINT stage1Height = (imgHeight / 4 > 0) ? imgHeight / 4 : 1;

UINT stage2Width = (imgWidth / 8 > 0) ? imgWidth / 8 : 1;
UINT stage2Height = (imgHeight / 8 > 0) ? imgHeight / 8 : 1;

// --- Создаем буфер Stage 1 (Меньше в 4 раза под блоки 4x4) ---
D3D11_TEXTURE2D_DESC descS1 = {};
descS1.Width = stage1Width;
descS1.Height = stage1Height;
descS1.MipLevels = 1;
descS1.ArraySize = 1;
descS1.Format = DXGI_FORMAT_R8G8B8A8_UNORM;
descS1.SampleDesc.Count = 1;
descS1.Usage = D3D11_USAGE_DEFAULT;
descS1.BindFlags = D3D11_BIND_RENDER_TARGET | D3D11_BIND_SHADER_RESOURCE;

ID3D11Texture2D* pTexS1 = NULL;
if (SUCCEEDED(g_pd3dDevice->CreateTexture2D(&descS1, NULL, &pTexS1))) {
    g_pd3dDevice->CreateRenderTargetView(pTexS1, NULL, &g_pStage1RTV);
    g_pd3dDevice->CreateShaderResourceView(pTexS1, NULL, &g_pStage1SRV);
    pTexS1->Release();
}

// --- Создаем буфер Stage 2 (Меньше в 8 раз под макроблоки 8x8) ---
D3D11_TEXTURE2D_DESC descS2 = {};

```

```
descS2.Width = stage2Width;
descS2.Height = stage2Height;
descS2.MipLevels = 1;
descS2.ArraySize = 1;
descS2.Format = DXGI_FORMAT_R8G8B8A8_UNORM;
descS2.SampleDesc.Count = 1;
descS2.Usage = D3D11_USAGE_DEFAULT;
descS2.BindFlags = D3D11_BIND_RENDER_TARGET | D3D11_BIND_SHADER_RESOURCE;
```

```
ID3D11Texture2D* pTexS2 = NULL;
```

```
if (SUCCEEDED(g_pd3dDevice->CreateTexture2D(&descS2, NULL, &pTexS2))) {
    g_pd3dDevice->CreateRenderTargetView(pTexS2, NULL, &g_pStage2RTV);
    g_pd3dDevice->CreateShaderResourceView(pTexS2, NULL, &g_pStage2SRV);
    pTexS2->Release();
}
```

```
// --- Создаем буфер Stage 3 (ПОЛНОРАЗМЕРНЫЙ под гладкий финальный кадр) ---
```

```
// --- ИСПРАВЛЕНИЕ: ПЕРЕВОДИМ STAGE 3 В НАСТОЯЩИЙ HDR-ФОРМАТ С ПЛАВАЮЩЕЙ
ТОЧКОЙ ---
```

```
D3D11_TEXTURE2D_DESC descS3 = {};
```

```
descS3.Width = imgWidth;
```

```
descS3.Height = imgHeight;
```

```
descS3.MipLevels = 1;
```

```
descS3.ArraySize = 1;
```

```
descS3.Format = DXGI_FORMAT_R16G16B16A16_FLOAT; // ВКЛЮЧИЛИ 16-БИТНЫЙ FLOAT ДЛЯ
МАТЕМАТИКИ ИЗ 2011 ГОДА!
```

```
descS3.SampleDesc.Count = 1;
```

```
descS3.Usage = D3D11_USAGE_DEFAULT;
```

```
descS3.BindFlags = D3D11_BIND_RENDER_TARGET | D3D11_BIND_SHADER_RESOURCE;
```

```
ID3D11Texture2D* pTexS3 = NULL;
```

```
if (SUCCEEDED(g_pd3dDevice->CreateTexture2D(&descS3, NULL, &pTexS3))) {
    g_pd3dDevice->CreateRenderTargetView(pTexS3, NULL, &g_pStage3RTV);
}
```

```

        g_pd3DDevice->CreateShaderResourceView(pTexS3, NULL, &g_pStage3SRV);
        pTexS3->Release();
    }
}
}

// === ИСПРАВЛЕНИЕ: ЖЕСТКОЕ И БЕЗОПАСНОЕ ИЗМЕНЕНИЕ РАЗМЕРОВ ОКНА ПРИ ОТКРЫТИИ
ФАЙЛА ===
if (SUCCEEDED(hr))
{
    HWND hMainWnd = FindWindowW(L"MyShaderVideoPlayerClass", NULL);
    if (hMainWnd)
    {
        // Рассчитываем габариты окна с учетом меню и заголовка Windows
        RECT rc = { 0, 0, (LONG)imgWidth, (LONG)imgHeight };
        AdjustWindowRect(&rc, WS_OVERLAPPEDWINDOW, TRUE);

        // Физически раздвигаем окно под размер картинки!
        MoveWindow(hMainWnd, 100, 100, rc.right - rc.left, rc.bottom - rc.top, TRUE);
    }
}

// =====
// НАСТРОЙКА ДИАПАЗОНОВ С КРОЛЛИНГА ПОД РАЗМЕР КАРТИНКИ
// =====
if (SUCCEEDED(hr))
{
    // Сохраняем размеры файла для обработчика прокрутки
    g_currentImgWidth = imgWidth;
    g_currentImgHeight = imgHeight;
    g_scrollX = 0;
    g_scrollY = 0;

    // === АВТОМАТИЧЕСКИЙ РАСЧЕТ КОЭФФИЦИЕНТОВ СКОРОСТИ СКРОЛЛИНГА ===

```

```

g_scrollMultiplierX = (imgWidth > (UINT)g_wndW) ? ((float)imgWidth / g_wndW) : 1.0f;
g_scrollMultiplierY = (imgHeight > (UINT)g_wndH) ? ((float)imgHeight / g_wndH) : 1.0f;

HWND hMainWnd = FindWindowW(L"MyShaderVideoPlayerClass", NULL);

if (hMainWnd)
{
    // Настраиваем горизонтальный ползунок (X)
    SCROLLINFO siX = {};
    siX.cbSize = sizeof(SCROLLINFO);
    siX.fMask = SIF_RANGE | SIF_PAGE | SIF_POS;
    siX.nMin = 0;
    siX.nMax = imgWidth;
    siX.nPage = 800; // Шаг страницы равен размеру окна
    siX.nPos = 0;
    SetScrollInfo(hMainWnd, SB_HORZ, &siX, TRUE);

    // Настраиваем вертикальный ползунок (Y)
    SCROLLINFO siY = {};
    siY.cbSize = sizeof(SCROLLINFO);
    siY.fMask = SIF_RANGE | SIF_PAGE | SIF_POS;
    siY.nMin = 0;
    siY.nMax = imgHeight;
    siY.nPage = 600;
    siY.nPos = 0;
    SetScrollInfo(hMainWnd, SB_VERT, &siY, TRUE);
}
}

// === ЗОЛОТОЕ ПРАВИЛО: ЧИСТО ЗАКРЫВАЕМ ВСЕ РЕСУРСЫ И ПАМЯТЬ БЕЗ УТЕЧЕК! ===
delete[] pPixelsBuffer;
pConverter->Release();
pFrame->Release();

```

```

pDecoder->Release();
pWICFactory->Release();

return hr;
}

// === НОВЫЙ КОД: ФУНКЦИЯ ВЫЗОВА СИСТЕМНОГО ПРОВОДНИКА WINDOWS ===
bool OpenFileDialog(HWND hwnd, bool bOpenVideo)
{
    OPENFILENAMEW ofn = {};
    wchar_t szFile[MAX_PATH] = L"";

    ofn.lStructSize = sizeof(ofn);
    ofn.hwndOwner = hwnd;
    ofn.lpstrFile = szFile;
    ofn.nMaxFile = sizeof(szFile) / sizeof(szFile[0]);

    if (bOpenVideo)
    {
        // Фильтр файлов для видеопотока
        ofn.lpstrFilter = L"Видео файлы (AVI, MP4)\0*.avi;*.mp4\0Все файлы (*.*)\0*.*\0";
        ofn.lpstrTitle = L"Выберите видео поток для дефектоскопа";
    }
    else
    {
        // Фильтр файлов для отдельных изображений
        ofn.lpstrFilter = L"Изображения (BMP, PNG)\0*.bmp;*.png\0Все файлы (*.*)\0*.*\0";
        ofn.lpstrTitle = L"Выберите кадр/картинку для анализа";
    }

    ofn.Flags = OFN_PATHMUSTEXIST | OFN_FILEMUSTEXIST;

```

```

// Распахиваем системное окно Проводника
if (GetOpenFileNameW(&ofn))
{
    // Если пользователь выбрал файл и нажал "Открыть" — копируем путь в нашу глобальную
    переменную
    wcsncpy_s(g_szSelectedFilePath, MAX_PATH, ofn.lpstrFile);
    return true;
}

return false; // Если пользователь нажал "Отмена"
}

// === КОНЕЦ НОВОГО КОДА ===

// Главная точка входа Windows-приложения
int WINAPI wWinMain(_In_ HINSTANCE hInstance, _In_opt_ HINSTANCE hPrevInstance, _In_ PWSTR
pCmdLine, _In_ int nCmdShow)
{
    // === НОВЫЙ КОД: ИНИЦИАЛИЗАЦИЯ СИСТЕМНОЙ БИБЛИОТЕКИ COM ДЛЯ ДВИЖКА WIC ===
    (void)CoInitializeEx(NULL, COINIT_APARTMENTTHREADED); // Запускает поддержку WIC в Windows

    const wchar_t CLASS_NAME[] = L"MyVideoPlayerWindowClass";

    WNDCLASS wc = { };
    wc.lpfnWndProc = WindowProc;
    wc.hInstance = hInstance;
    wc.lpszClassName = CLASS_NAME;
    wc.hCursor = LoadCursor(NULL, IDC_ARROW);
    RegisterClass(&wc);

    HWND hwnd = CreateWindowEx(0, CLASS_NAME, L"Мой Шейдерный Видеоплеер",
        WS_OVERLAPPEDWINDOW | WS_HSCROLL | WS_VSCROLL,
        CW_USEDEFAULT, CW_USEDEFAULT,
        800, 600,
        NULL, NULL, hInstance, NULL);

```

```
if (hwnd == NULL) return 0;
```

```
ShowWindow(hwnd, nCmdShow);
```

```
CreateAppMenu(hwnd); // НОВОЕ: Физически включаем меню на экране сразу после показа окна
```

```
if (FAILED(InitDevice(hwnd)))
```

```
{
```

```
    CleanupDevice();
```

```
    CoUninitialize(); // НОВОЕ: Чисто закрываем и освобождаем ресурсы COM-потока перед  
    выходом!
```

```
    return 0;
```

```
}
```

```
MSG msg = { };
```

```
while (WM_QUIT != msg.message)
```

```
{
```

```
    // === О Т Л А Д О Ч Н Ы Й   М А Я Ч О К ===
```

```
    static bool bLoopStart = true;
```

```
    if (bLoopStart) {
```

```
        //MessageBoxW(hwnd, L"Маячок Ц-1: Мы успешно вошли в бесконечный цикл обработки  
        сообщений wWinMain!", L"Отладка", MB_OK);
```

```
        bLoopStart = false;
```

```
    }
```

```
    if (PeekMessage(&msg, NULL, 0, 0, PM_REMOVE))
```

```
    {
```

```
        TranslateMessage(&msg);
```

```
        DispatchMessage(&msg);
```

```
    }
```

```
    else
```

```
    {
```

```
        Render();
```

```

        //RenderSplit(); // Переключили плеер в режим сравнения "До / После"
    }
}

CleanupDevice();

CoUninitialize(); // НОВОЕ: Чисто закрываем и освобождаем ресурсы СОМ-потока перед
выходом!

return 0;
}

// === ИСТИННАЯ ПРОГРАММНАЯ ОБРАБОТКА ИЗ 2011 ГОДА НА ЦЕНТРАЛЬНОМ ПРОЦЕССОРЕ
(CPU) ===

// === ИСТИННАЯ ПИРАМИДАЛЬНАЯ ФИЛЬТРАЦИЯ 2011 ГОДА НА ПРОЦЕССОРЕ (CPU) ===

HRESULT ApplyCpuFilter(unsigned int* pSrcPixels, unsigned int width, unsigned int height, unsigned int*
pOutPixels)
{
    if (!pSrcPixels || !pOutPixels || width == 0 || height == 0) return E_INVALIDARG;

    // Весовые коэффициенты яркости монохрома из 2011 года
    const float W_R = 0.299f;
    const float W_G = 0.587f;
    const float W_B = 0.114f;

    // -----
    // МАССИВ p1: Переводим исходный кадр BGRA в монохромную матрицу яркости
    // -----

    float* p1 = new float[width * height];
    for (unsigned int i = 0; i < width * height; i++)
    {
        unsigned int pixel = pSrcPixels[i];
        BYTE b = (pixel & 0x000000FF);
        BYTE g = ((pixel & 0x0000FF00) >> 8);
        BYTE r = ((pixel & 0x00FF0000) >> 16);
    }
}

```

```

    p1[i] = (float)r * W_R + (float)g * W_G + (float)b * W_B;
}

// -----
// МАССИВ p2: Первый уровень пирамиды (сжатие блоков 4x4 пикселя)
// -----

unsigned int w2 = width / 4;
unsigned int h2 = height / 4;
if (w2 == 0) w2 = 1;
if (h2 == 0) h2 = 1;

float* p2 = new float[w2 * h2];

// Цикл свертки блоков 4x4
for (unsigned int y = 0; y < h2; y++)
{
    for (unsigned int x = 0; x < w2; x++)
    {
        float sum = 0.0f;
        // Сканируем квадрат 4x4 пикселя из исходного массива p1
        for (unsigned int block_y = 0; block_y < 4; block_y++)
        {
            unsigned int src_y = y * 4 + block_y;
            if (src_y >= height) src_y = height - 1; // Защита от выхода за край

            for (unsigned int block_x = 0; block_x < 4; block_x++)
            {
                unsigned int src_x = x * 4 + block_x;
                if (src_x >= width) src_x = width - 1;

                sum += p1[src_y * width + src_x];
            }
        }
    }
}

```

```

    }

}

p2[y * w2 + x] = sum / 16.0f; // Среднее арифметическое блока 4x4
}
}

// -----
// МАССИВ p3: Второй уровень пирамиды (укрупнение блоков в 8x8, или 2x2 от p2)
// -----

unsigned int w3 = w2 / 2;
unsigned int h3 = h2 / 2;
if (w3 == 0) w3 = 1;
if (h3 == 0) h3 = 1;

float* p3 = new float[w3 * h3];

// Цикл свертки блоков 2x2 от предыдущего уровня p2
for (unsigned int y = 0; y < h3; y++)
{
    for (unsigned int x = 0; x < w3; x++)
    {
        float sum = 0.0f;
        for (unsigned int block_y = 0; block_y < 2; block_y++)
        {
            unsigned int src_y = y * 2 + block_y;
            if (src_y >= h2) src_y = h2 - 1;

            for (unsigned int block_x = 0; block_x < 2; block_x++)
            {
                unsigned int src_x = x * 2 + block_x;
                if (src_x >= w2) src_x = w2 - 1;
            }
        }
    }
}

```

```

        sum += p2[src_y * w2 + src_x];
    }
}
p3[y * w3 + x] = sum / 4.0f; // Среднее арифметическое макроблока
}
}

// -----
// СБОРКА И ФИНАЛЬНЫЙ ВЫВОД (Пока выводим промежуточный массив p1)
// -----

// Цикл высокоточного межполукадрового анализа и интерполяции контуров
for (unsigned int y = 0; y < height; y++)
{
    for (unsigned int x = 0; x < width; x++)
    {
        unsigned int idx = y * width + x;
        float src_gray = p1[idx]; // Исходная точка кадра

        // 1. ИНТЕРПОЛЯЦИЯ: Находим координаты текущей точки в укрупненных массивах p2 и p3
        unsigned int x2 = x / 4;
        unsigned int y2 = y / 4;
        if (x2 >= w2) x2 = w2 - 1;
        if (y2 >= h2) y2 = h2 - 1;
        float macro_mid = p2[y2 * w2 + x2]; // Средняя яркость блока 4x4

        unsigned int x3 = x / 8;
        unsigned int y3 = y / 8;
        if (x3 >= w3) x3 = w3 - 1;
        if (y3 >= h3) y3 = h3 - 1;
        float global_mid = p3[y3 * w3 + x3]; // Глобальный фон макроблока 8x8

        // 2. ВЫЧИСЛЕНИЕ ДИСПЕРСИИ И ЛОКАЛЬНОГО КОНТРАСТА

```

```

float delta = src_gray - macro_mid;

// Внутренний сдвиг локальной разницы (аналог вашего макробарьера p4)
float local_dispersion = abs(macro_mid - global_mid);

// Защитный знаменатель против клякс и деления на ноль из 2011 года
float denom = local_dispersion + 12.75f;
float local_contrast = delta / denom;

// Жесткое ограничение диапазона среза, как в HLSL шейдере
if (local_contrast < -1.5f) local_contrast = -1.5f;
if (local_contrast > 1.5f) local_contrast = 1.5f;

// 3. ФИНАЛЬНЫЙ СИНТЕЗ И КОРРЕКЦИЯ ДЕФЕКТОСКОПА
float out_gray = macro_mid + local_contrast * (local_dispersion * 1.8f);

// Нормализация яркости в стандартный диапазон [0.0, 255.0]
if (out_gray < 0.0f) out_gray = 0.0f;
if (out_gray > 255.0f) out_gray = 255.0f;

// Честная гамма-коррекция 2.2 силами центрального процессора
float norm_gray = out_gray / 255.0f;
float corrected_gray = pow(norm_gray, 1.0f / 2.2f);

// Проявление скрытых микроконтуров
corrected_gray = corrected_gray * 1.05f - 0.02f;
if (corrected_gray < 0.0f) corrected_gray = 0.0f;
if (corrected_gray > 1.0f) corrected_gray = 1.0f;

// Собираем готовый пиксель обратно в формат BGRA для вывода шторки
BYTE final_byte = (BYTE)(corrected_gray * 255.0f);
pOutPixels[idx] = (0xFF000000) | (final_byte << 16) | (final_byte << 8) | final_byte;

```

```
}  
}
```

```
// Чистое освобождение динамической памяти из ОЗУ компьютера
```

```
delete[] p1;
```

```
delete[] p2;
```

```
delete[] p3;
```

```
return S_OK;
```

```
}
```

```
// Главная точка входа Windows-приложения
```

```
LRESULT CALLBACK WindowProc(HWND hwnd, UINT uMsg, WPARAM wParam, LPARAM lParam)
```

```
{
```

```
    switch (uMsg)
```

```
    {
```

```
        case WM_HSCROLL:
```

```
        {
```

```
            SCROLLINFO si = {};
```

```
            si.cbSize = sizeof(SCROLLINFO);
```

```
            si.fMask = SIF_ALL;
```

```
            GetScrollInfo(hwnd, SB_HORZ, &si);
```

```
            int oldPos = si.nPos;
```

```
            switch (LOWORD(wParam))
```

```
            {
```

```
                case SB_LINELEFT: si.nPos -= 20; break; // Кликнули на левую стрелочку
```

```
                case SB_LINERIGHT: si.nPos += 20; break; // Кликнули на правую стрелочку
```

```
                case SB_PAGELEFT: si.nPos -= si.nPage; break;
```

```
                case SB_PAGERIGHT: si.nPos += si.nPage; break;
```

```
                case SB_THUMBTRACK: si.nPos = si.nTrackPos; break; // Тащат ползунок мышкой
```

```

}

// Зажимаем позицию в физические рамки картинки
if (si.nPos < 0) si.nPos = 0;
if (si.nPos > (int)(g_currentImgWidth - si.nPage)) si.nPos = (int)(g_currentImgWidth - si.nPage);

if (si.nPos != oldPos)
{
    g_scrollX = si.nPos;

    // ВАША ФУНДАМЕНТАЛЬНАЯ ФОРМУЛА ДОЛИ:
    float denominatorX = (float)(si.nMax - si.nMin);
    g_scrollRatioX = (denominatorX > 0.0f) ? (float)(si.nPos - si.nMin) / denominatorX : 0.0f;

    SetScrollInfo(hwnd, SB_HORZ, &si, TRUE);
    Render();
}
}

return 0;

case WM_VSCROLL:
{
    SCROLLINFO si = {};
    si.cbSize = sizeof(SCROLLINFO);
    si.fMask = SIF_ALL;
    GetScrollInfo(hwnd, SB_VERT, &si);

    int oldPos = si.nPos;
    switch (LOWORD(wParam))
    {
        case SB_LINEUP: si.nPos -= 20; break; // Кликнули на верхнюю стрелочку
        case SB_LINEDOWN: si.nPos += 20; break; // Кликнули на нижнюю стрелочку
    }
}

```

```

case SB_PAGEUP: si.nPos -= si.nPage; break;
case SB_PAGEDOWN: si.nPos += si.nPage; break;
case SB_THUMBTRACK: si.nPos = si.nTrackPos; break; // Тащат ползунок мышкой
}

if (si.nPos < 0) si.nPos = 0;
if (si.nPos > (int)(g_currentImgHeight - si.nPage)) si.nPos = (int)(g_currentImgHeight - si.nPage);

if (si.nPos != oldPos)
{
    g_scrollY = si.nPos;

    // ВАША ФУНДАМЕНТАЛЬНАЯ ФОРМУЛА ДОЛИ:
    float denominatorY = (float)(si.nMax - si.nMin);
    g_scrollRatioY = (denominatorY > 0.0f) ? (float)(si.nPos - si.nMin) / denominatorY : 0.0f;

    SetScrollInfo(hwnd, SB_VERT, &si, TRUE);
    Render();
}

}

return 0;

case WM_SIZE:
    if (g_pSwapChain != NULL)
    {
        UINT width = LOWORD(IParam);
        UINT height = HIWORD(IParam);

        if (width > 0 && height > 0)
        {
            if (g_pRenderTargetView) { g_pRenderTargetView->Release(); g_pRenderTargetView = NULL; }

```

```

// Перестраиваем буферы SwapChain под новое разрешение
g_pSwapChain->ResizeBuffers(0, width, height, DXGI_FORMAT_UNKNOWN, 0);

ID3D11Texture2D* pBackBuffer = NULL;
g_pSwapChain->GetBuffer(0, __uuidof(ID3D11Texture2D), (LPVOID*)&pBackBuffer);
if (pBackBuffer)
{
    g_pd3dDevice->CreateRenderTargetView(pBackBuffer, NULL, &g_pRenderTargetView);
    pBackBuffer->Release();
}

// Вызываем стандартную перерисовку
Render();
}
}
break;

```

```

// === НОВЫЙ КОД: ОБРАБОТКА НАЖАТИЙ НА ПУНКТЫ МЕНЮ ===
case WM_COMMAND:
    // В wParam система Windows передает ID нажатого пункта меню
    switch (LOWORD(wParam))
    {
        case IDM_VIEW_CPU:
            // 1. Инвертируем глобальный флаг
            g_bUseCPUProcessing = !g_bUseCPUProcessing;

            // 2. Ставим галочку в меню, используя точный идентификатор кнопки
            CheckMenuItem(GetMenu(hwnd), IDM_VIEW_CPU, g_bUseCPUProcessing ? MF_CHECKED : MF_UNCHECKED);

```

```
// 3. Командуем окну немедленно перерисовать кадр
```

```
InvalidateRect(hwnd, NULL, FALSE);
```

```
return 0;
```

```
// === НАСТОЯЩЕЕ ВНЕДРЕНИЕ: ОБРАБОТКА НАЖАТИЙ НА МАСШТАБЫ ДЕФЕКТОСКОПА ===
```

```
case IDM_ZOOM_FIT: g_bFitToWindow = true; g_zoomDivider = 1.0f; goto my_reset_scroll;
```

```
case IDM_ZOOM_1: g_bFitToWindow = false; g_zoomDivider = 1.0f; goto my_reset_scroll;
```

```
case IDM_ZOOM_2: g_bFitToWindow = false; g_zoomDivider = 2.0f; goto my_reset_scroll;
```

```
case IDM_ZOOM_4: g_bFitToWindow = false; g_zoomDivider = 4.0f; goto my_reset_scroll;
```

```
case IDM_ZOOM_8: g_bFitToWindow = false; g_zoomDivider = 8.0f; goto my_reset_scroll;
```

```
case IDM_ZOOM_16: g_bFitToWindow = false; g_zoomDivider = 16.0f; goto my_reset_scroll;
```

```
my_reset_scroll:
```

```
{
```

```
// 1. Возвращаем скроллинг в исходный левый верхний угол кадра
```

```
g_scrollX = 0; g_scrollY = 0;
```

```
g_scrollRatioX = 0.0f; g_scrollRatioY = 0.0f;
```

```
// 2. АВТОМАТИЧЕСКАЯ РАССТАНОВКА ГАЛОЧЕК (V) В МЕНЮ ОКНА
```

```
HMENU hViewMenuRef = GetSubMenu(GetMenu(hwnd), 1); // Находим наше меню "Вид"  
(индекс 1)
```

```
if (hViewMenuRef)
```

```
{
```

```
HMENU hZoomMenuRef = GetSubMenu(hViewMenuRef, 1); // Находим подменю "Масштаб"  
внутри "Вида"
```

```
if (hZoomMenuRef)
```

```
{
```

```
// Сбрасываем старые птички со всех 6 пунктов
```

```
CheckMenuItem(hZoomMenuRef, IDM_ZOOM_FIT, MF_UNCHECKED);
```

```
CheckMenuItem(hZoomMenuRef, IDM_ZOOM_1, MF_UNCHECKED);
```

```
CheckMenuItem(hZoomMenuRef, IDM_ZOOM_2, MF_UNCHECKED);
```

```
CheckMenuItem(hZoomMenuRef, IDM_ZOOM_4, MF_UNCHECKED);
```

```
CheckMenuItem(hZoomMenuRef, IDM_ZOOM_8, MF_UNCHECKED);
```

```
CheckMenuItem(hZoomMenuRef, IDM_ZOOM_16, MF_UNCHECKED);
```

```
// Включаем галочку строго на выбранном оператором режиме!
```

```
UINT activeID = IDM_ZOOM_FIT;
```

```
if (!g_bFitToWindow)
```

```
{
```

```
    if (g_zoomDivider == 1.0f) activeID = IDM_ZOOM_1;
```

```
    if (g_zoomDivider == 2.0f) activeID = IDM_ZOOM_2;
```

```
    if (g_zoomDivider == 4.0f) activeID = IDM_ZOOM_4;
```

```
    if (g_zoomDivider == 8.0f) activeID = IDM_ZOOM_8;
```

```
    if (g_zoomDivider == 16.0f) activeID = IDM_ZOOM_16;
```

```
}
```

```
CheckMenuItem(hZoomMenuRef, activeID, MF_CHECKED);
```

```
}
```

```
}
```

```
// 3. СИНХРОНИЗАЦИЯ ПОЛЗУНКОВ ПОД ВЫБРАННЫЙ МАСШТАБ
```

```
UINT currentMaxX = g_bFitToWindow ? (UINT)g_wndW : (UINT)((float)g_currentImgWidth /  
g_zoomDivider);
```

```
UINT currentMaxY = g_bFitToWindow ? (UINT)g_wndH : (UINT)((float)g_currentImgHeight /  
g_zoomDivider);
```

```
SCROLLINFO zoomSiX = {};
```

```
zoomSiX.cbSize = sizeof(SCROLLINFO);
```

```
zoomSiX.fMask = SIF_RANGE | SIF_PAGE | SIF_POS;
```

```
zoomSiX.nMin = 0;
```

```
zoomSiX.nMax = (currentMaxX > 0) ? currentMaxX : 1;
```

```
zoomSiX.nPage = (UINT)g_wndW;
```

```
zoomSiX.nPos = 0;
```

```
SetScrollInfo(hwnd, SB_HORZ, &zoomSiX, TRUE);
```

```
SCROLLINFO zoomSiY = {};
```

```
zoomSiY.cbSize = sizeof(SCROLLINFO);
```

```

zoomSiY.fMask = SIF_RANGE | SIF_PAGE | SIF_POS;

zoomSiY.nMin = 0;

zoomSiY.nMax = (currentMaxY > 0) ? currentMaxY : 1;

zoomSiY.nPage = (UINT)g_wndH;

zoomSiY.nPos = 0;

SetScrollInfo(hwnd, SB_VERT, &zoomSiY, TRUE);


// 4. Перерисовываем экран

InvalidateRect(hwnd, NULL, FALSE);

return 0;
}


case IDM_FILE_EXIT:

    // Если нажали "Выход" — закрываем окно программы

    DestroyWindow(hwnd);

    return 0;


case IDM_HELP_ABOUT:

    // Если нажали "О программе" — показываем красивое информационное окно

    MessageBoxW(hwnd,

        L"Шейдерный Видеоплеер-Дефектоскоп v2026\n\n"

        L"Реализован пятипроходный вычислительный конвейер.\n"

        L"Алгоритмы фильтрации локального контраста и дисперсии макроблоков 4x4 и 8x8.\n\n"

        L"Разработчик математического ядра: Курчанов А. Ф.",

        L"О программе",

        MB_OK | MB_ICONINFORMATION);

    return 0;


case IDM_FILE_OPEN_VIDEO:

    // Вызываем Проводник в режиме фильтрации видео (передаем true)

    if (OpenFileDialog(hwnd, true))

    {

```

```

        // Если файл выбран успешно — выведем его путь на экран для проверки

        //MessageBoxW(hwnd, g_szSelectedFilePath, L"Видео поток успешно подключен", MB_OK |
        MB_ICONINFORMATION);

        // TODO: Здесь на следующем шаге мы запустим инициализацию Media Foundation
    }

    return 0;

case IDM_FILE_OPEN_IMAGE:

    // 1. Распахиваем Проводник для выбора картинки
    if (OpenFileDialog(hwnd, false))
    {
        // 2. Если файл выбран — загружаем его пиксели в Point-текстуру через WIC
        if (SUCCEEDED(LoadTextureFromFile(g_szSelectedFilePath)))
        {
            // 3.
            // Принудительно вызываем Render прямо сейчас,
            // чтобы протолкнуть новые SRV-интерфейсы в GPU!
            Render();

            //RenderSplit(); // Переключили плеер в режим сравнения "До / После"
        }
        else
        {
            MessageBoxW(hwnd, L"Не удалось декодировать файл изображения через WIC.",
            L"Ошибка дефектоскопа", MB_OK | MB_ICONERROR);
        }
    }

    return 0;

case IDM_FILE_SAVE_RESULT:

case IDM_VIEW_SPLIT:

case IDM_VIEW_RESULT_ONLY:

```

```
        // Для остальных пунктов пока выведем временную заглушку, чтобы видеть отклик  
        MessageBoxW(hwnd, L"Этот блок конвейера находится в режиме подключения данных.",  
L"Информация", MB_OK | MB_ICONINFORMATION);  
        return 0;  
    }  
    break;  
    // === КОНЕЦ НОВОГО КОДА ===  
  
case WM_DESTROY:  
    PostQuitMessage(0);  
    return 0;  
}  
return DefWindowProc(hwnd, uMsg, wParam, lParam);  
}
```